PROJECT REPORT

Finance Tracker

Helping a beginner see where their money goes, before the month runs out.

Apratim Saxena

apratim1424@gmail.com

Code: github.com/apratim-1407/finance_tracker

01 / SUMMARY

The short version

A small tool for a very common problem: people who earn a salary and still reach the end of the month wondering where it went.

What I built. Finance Tracker is a terminal app written in Python. You tell it your salary and a monthly budget, then log each expense with a category and a note. It shows what you have spent, what is left, and where the money went. It runs on any laptop with Python and needs no sign-up.

Why this is more than a coding project. The code is simple on purpose. The interesting part is the decisions behind it: who the tool is for, what I chose to leave out, and how I would know if it works. This document walks through that thinking.

> **A note on honesty**
> I have not run formal user research on this tool, so I do not quote usage numbers. Where I describe user needs, they are assumptions drawn from my own experience and conversations, and I mark them as such. The metrics in this document are the ones I would track, not results I have already measured.

The problem

Most people do not overspend on one big purchase. They overspend through many small ones they never added up.

What happens today. A student or first-job earner gets paid, spends through UPI, cards and cash, and checks the balance only when something fails at checkout. By then the decisions are already made. The information needed to avoid the problem exists, but it is scattered across SMS alerts, bank apps and memory.

Why the existing options do not fit a beginner.

- **Spreadsheets** are flexible, but setting one up takes effort most people give up on after a week.
- **Budgeting apps** ask for sign-ups, bank access and permissions. For someone just starting, that is a lot of trust to give up front.
- **Doing nothing** is the real competitor. It costs no effort today, so it usually wins.

Problem statement

> **In one sentence**
> A beginner who wants to control their spending needs a way to record an expense in a few seconds and see how much budget is left, without signing up for anything or learning a complicated tool.

Why a terminal app?

A terminal is not what a typical consumer would choose. I chose it for two reasons. First, it let me test the core idea (log, categorise, compare against budget) with the least possible build. Second, I wanted to learn the problem properly before spending time on screens and design. It is the equivalent of testing an idea on paper before building the app. The trade-off: anyone not comfortable with a terminal cannot use it, which shapes the next steps on page 8.

02 / THE USER

Who this is for, and what they are trying to do

I designed for one person first. A product that tries to serve everyone on day one usually serves no one well.

Primary user (an assumption, not a research finding)

	Description
Who	A college student or first-job earner, 18–26, with a fixed monthly income or allowance
Comfort level	Okay with simple tools; not looking for investment features or tax planning
Situation	Spends across UPI, cards and cash; has no single place that shows the total
Fear	Running out of money before the month ends, and not knowing why
What stops them	Setup effort, privacy worries about linking bank accounts, and loss of interest after a few days

What they are really trying to get done

I frame this as the "job" the user is hiring the product for, because it keeps the focus on the outcome and not the feature list.

Job	In the user's words	What the tool must do
Know where I stand	"How much can I still spend this month?"	Show total spent and budget left at any time
Remember what I spent	"I know I bought something, but what?"	Save each expense with amount, category, note and date
Spot the leak	"Why is my money gone already?"	Group spending by category and show the biggest items
Fix mistakes quickly	"I typed the wrong amount."	Let me delete an entry, with a confirmation step

What I would test first

These are the assumptions that, if wrong, would change the product the most:

- People will log an expense if it takes under 10 seconds.
- A single monthly budget is enough; they do not need to plan every week.
- Seeing the category breakdown changes behaviour more than seeing only the total.

03 / APPROACH

How I decided what to build

I followed a simple loop: define the problem, pick the smallest useful version, build it, then list what is still missing.

1	2	3	4	5
Define the user and their job	List every possible feature	Keep only what serves the core job	Build, then test by using it	Write down gaps and next steps

Deciding what goes in version 1

I listed ideas freely, then asked one question of each: *does the user need this to answer "how much can I still spend?"* If not, it waited.

Idea	Decision	Reason
Log expense with category, note, date	Build	Core of the product; without it nothing else works
Monthly budget and balance left	Build	This is the answer to the user's main question
Per-category spending limits	Build	Helps spot which category causes the overspend
View, search, filter, delete	Build	People make mistakes; a log they cannot fix loses trust
Save a summary file on exit	Build (basic)	Gives a record to keep, even if it cannot be reloaded yet
Bank or UPI auto-import	Not now	Needs permissions and trust; too heavy for a first version
Charts and graphs	Not now	Useful, but the numbers must be right first
Login and multiple users	Not now	One user on one device is enough to test the idea
Mobile app	Not now	Test the idea before investing in screens

Principle I followed

> **Smallest version that answers the question**
> A product that does three things reliably is more useful than one that does ten things badly. Every feature I cut is a feature I do not have to maintain, explain or fix.

04 / THE PRODUCT

What each feature is for

Every feature exists to solve a specific user problem. Here is the link between the two.

The core flow

1	2	3	4	5
Enter name, salary and monthly budget	See the menu	Add, view, search or delete	Check spent vs. budget left	Exit and save a summary

Feature	User problem it solves	Design decision
Welcome by name and budget setup	Tool feels generic and the user has no starting limit	Ask only three questions at the start, so the user reaches value quickly
Add expense (amount, category, note, auto-date)	Logging feels like effort, so people stop	The date fills itself. Note is optional. Fewer fields means more entries
Category list plus custom categories	Fixed categories never match real life	Sensible defaults (Food, Travel, Bills, Shopping, Others) and the freedom to add more
Per-category limits	The total looks fine while one category quietly overshoots	Optional, so a beginner is not forced to plan in detail
View and search entries	User cannot recall what a charge was	Keyword search covers both description and note
Filter by category or day	Hard to see where money concentrates	Daily view answers “what did I spend today?”
Delete with confirmation	Typing mistakes are common and permanent errors destroy trust	Ask “are you sure?” before removing anything
Highest, lowest and average spend	Total alone hides the one big purchase	Simple numbers, no jargon
Reset for a new month	Old data clutters the new month	Explicit action, so data is never wiped by accident
Summary saved to a text file	User wants a record outside the app	Plain text opens anywhere, no special software

05 / DECISIONS

Trade-offs I made, and why

Product work is mostly choosing between two good options. These were the main choices.

Decision	Option chosen	What I gave up
Interface	Text menu in a terminal	Reach. People who do not use a terminal cannot use it. Gained speed of building and learning.
Where data lives	In memory while the app runs, plus a summary text file	Persistence. Closing the app loses the live data. Gained simplicity and no privacy risk.
Data entry	Manual typing	Convenience. Auto-import would be easier but needs bank trust. Gained control and zero permissions.
Setup	Three questions at start	Depth. The tool knows little about the user. Gained a fast first use.
Budget model	One monthly budget, plus optional category limits	Precision. No weekly planning. Gained an easy mental model.
Code structure	Linear code with no functions	Maintainability. Harder to extend. Gained readability for a first project.

The decision I would revisit first

Persistence. Right now the tool writes a readable summary when you exit, but it cannot load it back. A budgeting tool that forgets everything each time you close it does not build a habit. It is the biggest gap between what I built and what a user would keep using.

I accepted this in version 1 because the goal was to check whether the log-and-compare flow made sense. It is the first thing I would fix, and page 8 explains where it sits in the plan.

How I handled bad input

A tool that crashes on a typo teaches users to stop trusting it. I added basic checks so wrong input gets a clear message and a second chance: numbers must be numbers, category choices must exist, and an invalid category falls back to “Misc” instead of losing the entry. It is not complete (page 7), but it follows one rule: *never lose what the user just typed*.

06 / EVALUATION

What is weak today, and how I would know it works

Honest gaps

Gap	Why it matters	Severity
Live data cannot be reloaded after closing	Breaks the habit the product depends on	High
Only works in a terminal	Excludes most of the intended users	High
Input checks are basic	Unusual input can still cause errors or wrong totals	Medium
No alerts when a limit is close	The user finds out after overspending, not before	Medium
No graphs or trends	Hard to see patterns across weeks	Low for now

How I would measure success

These are metrics I would track once real people use it. They are targets to learn from, not results so far.

Question	Metric	What a good sign looks like
Do people get started?	Share of new users who log at least one expense in their first session	Most users reach their first entry
Is logging easy?	Time from opening the app to saving an expense	Under 10 seconds after the first time
Do they come back?	Users who log expenses in at least 3 separate weeks	Rising week over week
Does it help?	Users who stay inside their budget compared with their own earlier months	More months on budget
Is it trusted?	Deletes and corrections per 100 entries; errors per session	Low and falling

The one number I would put first
Weekly active loggers: people who recorded at least one expense this week. A budgeting tool only works if people keep using it, so repeat use matters more than sign-ups or total entries. I would watch it alongside a guardrail: if corrections per 100 entries rises, new features are making the tool harder to use.

07 / NEXT STEPS

What I would build next, and what I learned

Prioritised roadmap

I rank by how much each item helps the user (impact) against how much work it needs (effort). Highest impact for the least effort goes first.

Order	Item	Impact	Effort	Why here
1	Save data to a file and load it on start	High	Low	Fixes the biggest gap; makes the tool a habit
2	Warn when spending nears a limit	High	Low	Moves from reporting to helping in the moment
3	Stronger input checks	Medium	Low	Protects trust
4	Weekly summary by category	Medium	Medium	Shows the pattern, not just the total
5	Simple web or phone screen	High	High	Opens the product to non-technical users; worth it once 1–4 work
6	Auto-import from SMS or UPI	High	High	Removes typing; needs a trust and privacy plan first

How I would test before building 5 and 6. I would ask ten students to use the file-saving version for two weeks. If most of them log in at least three different weeks, the habit is real and a proper app is worth building. If not, I would go back to the problem and not to the interface.

What I learned

- **Scope is a decision.** Each cut feature made the tool easier to finish, explain and fix.
- **The weak spot is a product issue, not a code issue.** Missing persistence does not break the code, it breaks the habit.
- **Small does not mean shallow.** Even a simple tool needs a clear user, a clear problem, and a way to tell whether it works.
- **Know what you do not know.** I have not tested with real users yet, so this document separates assumptions from facts.